

PDC Assignment 2 — Building Resilient Distributed Systems

Saleha Shoaib | BSCS23036 | Parallel and Distributed Computing | Repo: PDC-Sp24-BSCS23036-Shoaib

Part 1 — Analysis of the Naive Architecture

1. Lost Update (Synchronization)

Each edit is a read-modify-write (RMW) cycle: the client GETs document $v=42$, mutates a local copy, then PUTs the entire payload. The database row is updated atomically, but the *transaction boundary is per-write*, not across the user's edit session, so the two RMW cycles are not serializable. The DB has no precondition (no version column, no *SELECT FOR UPDATE*, no row-level lock taken at GET time), so the second write is a blind overwrite under last-writer-wins semantics. Concurrency control lives nowhere — not in the API, not in the data layer — which is the textbook lost-update / write-skew anomaly.

2. Dropped Cancellation Webhook (Coordination)

Clerk delivers *subscription.deleted* with at-most-once semantics from our backend's perspective: it is a single fire-and-forget HTTP POST. If the TCP connection drops or our handler 5xx's, the event is gone, the user record never flips to *is_premium=False*, and our DB is now permanently divergent from Clerk's source of truth. There is no idempotency key, no durable inbox, no retry policy, and no reconciliation sweep, so even if Clerk did retry we would either double-process or have no way to deduplicate. This is a Two Generals problem in miniature: two independent services with no agreement protocol, exchanging exactly one lossy message.

3. Synchronous LLM Call (Fault Tolerance)

The /chat handler issues a blocking outbound call with no per-request deadline (it inherits the upstream's 60 s socket timeout). Because the FastAPI worker is occupied for that entire window, the upstream's failure mode propagates into our process: under load the worker pool fills with stuck requests and queue depth on every *other* endpoint balloons. Health checks, login, even static assets stop responding. There is no bulkhead isolating the LLM client, no circuit to short-circuit the broken dependency, and no fallback — so one faulty dependency causes a cascading failure across the whole service.

Part 2 — Proposed Architecture

2.1 Synchronization — Optimistic Locking with Versioning

Add a monotonic *version* column to *documents*. Every read returns the version; every write sends it back as a precondition. The server commits with a single atomic compare-and-swap:

```
UPDATE documents SET body=:b, version=version+1 WHERE id=:id AND version=:client_version;
```

If *rowcount=0*, the row was changed by someone else → respond **409 Conflict** with the current server version; the client refetches, merges (3-way diff or OT/CRDT for collaborative editors), and retries. Pessimistic locking (*SELECT FOR UPDATE*) would also work but holds DB locks across user think-time, so it scales worse and creates head-of-line blocking. Optimistic locking is wait-free in the common case.

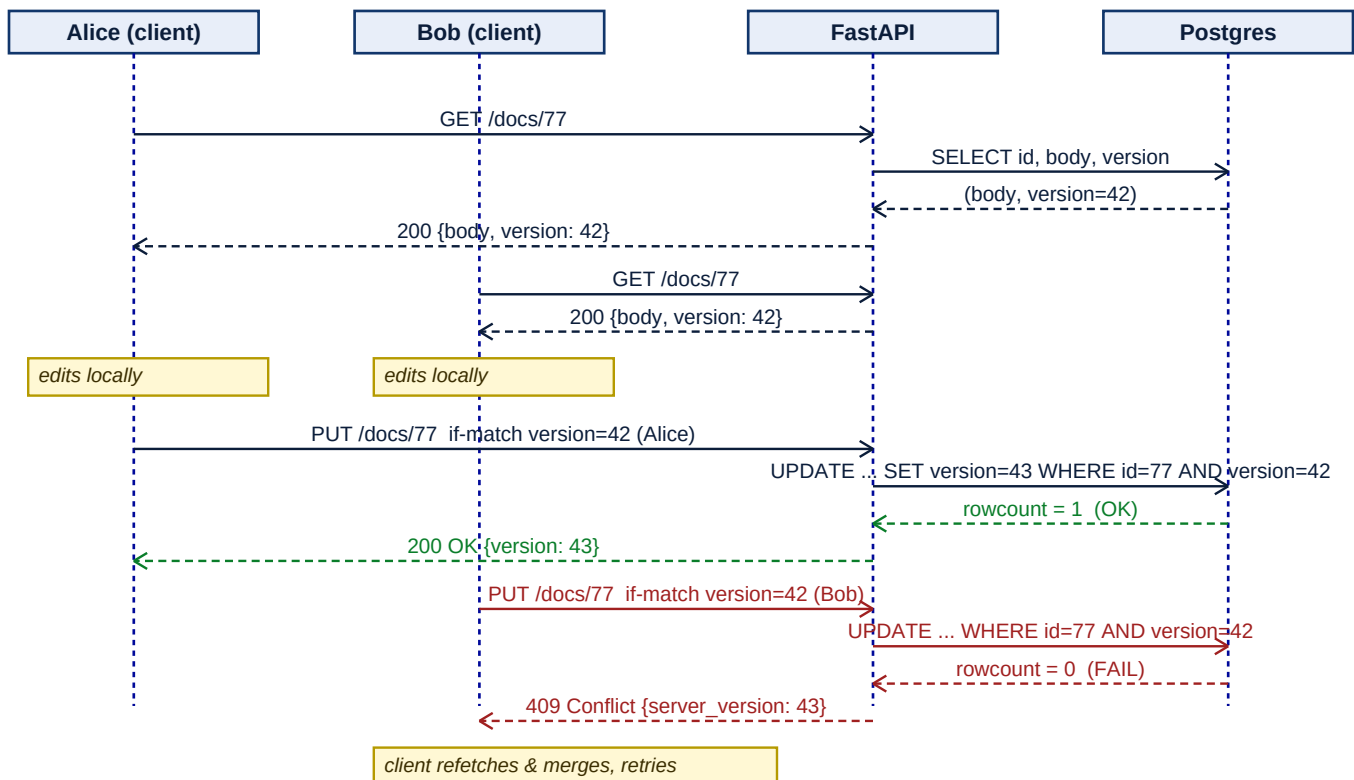


Figure 1. UML sequence diagram for the optimistic-locking sync fix. Both clients read version 42; Alice's CAS succeeds (rowcount=1) and the version becomes 43; Bob's stale CAS fails (rowcount=0) and the server returns 409 with the current version so the client can merge.

2.2 Coordination — Idempotent Inbox + Retries + DLQ

Make webhook delivery *at-least-once* and processing *idempotent*:

- Clerk attaches a unique *Svix-Id* per event. The handler **INSERTs** it into a *webhook_inbox*(*svix_id* PRIMARY KEY) table inside the same transaction that applies the state change; on duplicate-key the handler short-circuits with 200 OK, so safe retries are free.
- Persist the event to a durable queue (Postgres + *SKIP LOCKED*, Redis Streams, or RabbitMQ quorum queues) and **ack only after** the DB transaction commits. A worker pool drains the queue with bounded exponential backoff (e.g. 1 s, 5 s, 30 s, 5 min, ...).
- After N attempts the message lands in a Dead Letter Queue and pages an operator. A nightly reconciliation job calls Clerk's billing API and repairs any drift — the convergence safety net.
- End-to-end this is at-least-once delivery + idempotent application = effectively exactly-once for our state, without distributed transactions.

2.3 Fault Tolerance — Circuit Breaker + Fallback

Wrap the LLM client in a CLOSED → OPEN → HALF_OPEN circuit breaker:

- **Per-call timeout** (e.g. 2 s), independent of the upstream's 60 s socket timeout, so a single hung request never holds a worker hostage.
- After K consecutive failures the breaker **OPENS** for T seconds; during that window every call **fails fast** with no upstream I/O — the broken dependency is isolated.
- After T elapses, one probe is admitted in **HALF_OPEN**; success → CLOSED, failure → OPEN again. This prevents a thundering herd from re-killing a recovering upstream.
- On every failure path the handler returns a **fallback** (cached prior answer, smaller local model, or templated "tutor is degraded" message) instead of bubbling the exception.
- Optionally pair with a **bulkhead** (semaphore on concurrent LLM calls) so the LLM cannot starve unrelated endpoints' workers.

This is the pattern implemented in *app/circuit_breaker.py* and verified by *tests/test_circuit_breaker.py*.

2.4 CAP / PACELC Trade-offs

Partition tolerance is non-negotiable in a real system, so per Brewer's theorem each component is really a choice between Consistency and Availability; PACELC adds a Latency vs Consistency axis when no partition is happening. The proposed architecture deliberately picks differently in different places:

- **Sync fix — CP / PC.** We pick Consistency: concurrent writers may be rejected with 409 (a small Availability hit on the write path) to guarantee no lost update. The latency cost is one extra SQL predicate, which is negligible. This is the right call for user-authored data, where silent corruption is unacceptable.
- **Coordination fix — AP / EL.** The auth provider and our DB may briefly disagree during a retry storm. We accept that bounded staleness in exchange for staying available during partitions; the idempotent inbox + DLQ + nightly reconciliation guarantee eventual convergence. Webhook latency rises slightly (extra DB writes per event), which is fine because no human is waiting on it.
- **Fault-tolerance fix — AP / EL.** When the LLM is unhealthy we serve a stale or templated answer rather than block the user. We trade strict freshness for a 200 ms response instead of a 60 s hang and a process-wide outage. From the user's perspective this is a massive Availability and Latency win at the cost of correctness on a single, non-critical code path.

Net architectural philosophy: **CP at the core data plane, AP at the edges** (third-party integrations, ML calls). This matches how production systems at scale (Stripe, GitHub, Netflix) are actually built.